

POLITECNICO DI MILANO

CONSTRAINED NUMERICAL OPTIMIZATION FOR ESTIMATION AND CONTROL
A.Y. 2023/2024

Lecturer: prof. Lorenzo Mario Fagiano

Trajectory Optimization on the Yas Marina Circuit of an autonomous
racing car

Students:

Leonardo Polisini

Alexandra Spertini

Andrea Trebbi

Alex Zagati

Contents

Abstract	2
1 Introduction	3
2 Problem Description	4
2.1 2D model of the track	4
2.1.1 Dynamical model of the track	4
2.1.2 Change of independent variable	5
2.2 Model of the vehicle	6
2.2.1 Tire Forces	6
2.2.2 Input of the system and torque constraints	9
3 Method	10
3.1 Modeling of the optimization problem	10
3.2 Discretization of the problem	11
3.2.1 Transcription	11
3.3 Scaling	16
4 Theoretical Background	17
4.1 Theory behind the solver	17
4.2 Solver choice	22
4.2.1 SQP algorithm - using BFGS	22
5 Results	24
5.0.1 Simple curve	24
5.0.2 Entire circuit	26
6 Discussion	31
6.1 Initial condition	31
6.2 Solver Performance	31
6.3 Application-related results	32
7 Conclusions	33
Appendix	34
7.1 Appendix A: track equations	34
References	36
Nomenclature	37

Abstract

This report provides an in-depth exploration of the determination of the optimal trajectory for a racing car on the Yas Marina Circuit in Abu Dhabi, with a primary focus on minimizing lap time. The optimization problem is rigorously defined as a constrained nonlinear optimization task, utilizing a single-track model of the vehicle. This last is selected for its ability to capture the key dynamic behaviors of the car while simplifying the problem by excluding complex factors such as load transfer and rolling resistance.

By focusing on these core dynamics, the study aims to develop an approach to trajectory optimization that is both efficient and computationally manageable. The goal is to gain meaningful insights into vehicle's performance limits under specific constraints, thereby enhancing our understanding of how to optimize the lap time.

To effectively address the optimization problem within the framework of this specific vehicle model, the study employs the *direct orthogonal collocation method* to discretize it. This technique is crucial for transforming the continuous optimization problem into a discrete one, making it more manageable for numerical computation. Additionally, the study emphasizes the importance of *scaling* the decision variables and constraints. This process is essential for improving the numerical stability and efficiency of the optimization algorithm, ensuring that the compiler can solve the problem more effectively.

We proceed by formulating the optimization problem as a discrete, nonlinear, and constrained one. To tackle this complex problem, we employ the Sequential Quadratic Programming (SQP) algorithm with *Broyden-Fletcher-Goldfarb-Shanno* (BFGS) algorithm for Hessian computation. This last works by iteratively updating an approximation of the Hessian matrix H , which represents the second-order partial derivatives of the objective function.

This study successfully achieves the objective of optimizing the trajectory of a racing car driving on the Yas Marina circuit.

Chapter 1

Introduction

The Minimum Lap Time Problem (MLTP) consists in the determination of the trajectory, intended as a path and the corresponding velocities, that minimizes the lap time of a racing car running along a given track. Extracting the minimum lap time characteristics for an autonomous racing car has multiple objectives: firstly, it can facilitate autonomous car races where the goal is to finish the competition in the minimum time possible, and secondly its outcomes may be eventually useful on public roads in case of emergency maneuvers.

It is possible to distinguish between two main approaches for the MLTP: a *geometric approach* and a *dynamic approach*. The former benefits of a simple and straightforward problem formulation, which is independent from the vehicle model adopted and allows to extract the optimal racing line shape and speed in two consecutive steps (optimal racing line determination and subsequent optimal speed profile association). However, this approach allows to reach up to $\sim 90\%$ accuracy of the optimal lap time. If the full performance exploitation is sought, the dynamic method comes into play. The higher precision is counterbalanced by a more complex formulation of the MLTP, which in turn leads to computationally expensive problems.

Of specific interest for this report is the Optimal Control Problem (OCP) formulation of the MLTP. Its core idea is to transform the problem into an OCP where the equations of motion becomes constraints. Subsequently, the methods for solving the corresponding OCP can be distinguished in dynamic programming, indirect methods and direct methods [3]. These last, in particular, translate the OCP into a NonLinear Program (NLP) through a process called direct transcription. This work investigates the use of *direct orthogonal collocation* in combination with a SQP solution algorithm for the problem solution.

In particular, **Chapter 2** defines the model of the track and of the circuit used for the optimization. **Chapter 3** describes the method used for the problem setting, with specific focus on the modeling of the optimization problem, the process of converting the OCP into a discrete NLP through *direct orthogonal local collocation* and the scaling.

Chapter 4 describes the theory behind the optimization routine used, implemented through a custom MATLAB function *myfmincon*, and details some solver parameter choices for the presented solution.

Chapter 5 contains the results of the obtained solution, with its advantages and limitations.

Chapter 6 is devoted to the discussion of the obtained solution in terms of solver performances and application-related results.

Conclusively, in **Chapter 7** we draw the conclusions and summarize the important outcomes from the study.

Chapter 2

Problem Description

The initial step in addressing the Minimum Lap Time Problem involves developing a comprehensive model encompassing the two components of the problem: the track and the vehicle.

The model of the track shall accurately represent the circuit, including detailed information on the track layout like turns, straight sections and width. The track model needs to capture all these elements to provide a realistic simulation environment for vehicle's performance analysis.

The second part focuses on the vehicle itself, where we employ a simplified yet effective representation, known as the single-track or bicycle model, that is able to reduce the complexity of vehicle's dynamics. Despite its simplicity, the bicycle model is highly effective for analyzing the dynamics of the car, as it captures essential aspects such as lateral and longitudinal dynamics, tire forces, and vehicle stability.

By integrating these two models, we create a simulation environment that accurately reflects both the track conditions and vehicle's dynamics. This combined model serves as the foundation for solving the MLTP, allowing us to explore various scenarios and optimize vehicle's performance to achieve the minimum lap time.

2.1 2D model of the track

2.1.1 Dynamical model of the track

This process begins by defining track's curvature and width. Specifically, the curvature of the track at any given point is represented by a parameter κ , which measures the curvature of the centerline of the track. In addition, track's width, denoted by w , is assumed to be constant throughout its length. To describe vehicle's position on the track accurately, we employ a curvilinear coordinate system. In this system, vehicle's position is represented by the curvilinear abscissa s of the centerline, which serves as the independent variable (see Figure 7.1). This approach allows us to translate track's shape and dimensions into a coordinate framework that aligns with vehicle's movement.

The precise location of the car on the track is defined using two important parameters. The first parameter n represents the normal distance between the car and the centerline of the track. This distance measures how far the car is displaced perpendicularly with respect to the centerline and it is meant positive when the displacement of vehicle's Center Of Gravity (COG) is from the centerline towards the outer line. The second parameter is the relative angle ϵ , which indicates the angle between car's direction of movement and the tangent to the centerline at car's current position.

Once these parameters are established, we can proceed to formulate the equations that describe vehicle's dynamics and its movement along the track. These equations will incorporate both the curvature of the track and vehicle's interaction with it. They are designed to capture how the car responds to changes in track's geometry and its own movement dynamics. By integrating them, we can analyze vehicle's behavior in various scenarios, predicting its path, stability, and performance. This comprehensive approach is essential for understanding how the vehicle behaves under different conditions.

$$\frac{ds}{dt} = \frac{v_x \cos \epsilon - v_y \sin \epsilon}{1 - n\kappa} \quad (2.1)$$

$$\frac{dn}{dt} = v_x \sin \epsilon + v_y \cos \epsilon \quad (2.2)$$

$$\frac{d\epsilon}{dt} = \Omega_z - \kappa \frac{ds}{dt} \quad (2.3)$$

In the dynamic model of the vehicle, t is the time variable, v_x and v_y represent respectively the longitudinal and lateral velocities of the vehicle in its local reference frame, while Ω_z denotes the yaw rate, which measures the rate at which vehicle's orientation changes.

Details on how equations (2.1)-(2.3) are derived can be found in Appendix 7.1.

2.1.2 Change of independent variable

Using time as independent variable in dynamic modeling can sometimes be less efficient than using the curvilinear abscissa s along the track. There are several reasons why adopting s as the independent variable can be advantageous.

Firstly, when the optimization problem, often referred to as the OCP, is formulated with s as the independent variable, it becomes easier to express the cost function and constraints along the circuit. By using s we align the discretization process directly with track's geometry, simplifying the problem. Secondly, when using s as the independent variable, we can directly define the curvature and width of the track at each discretization point. If time were used as the independent variable, the relationship between the vehicle's position and these parameters would be implicit. This would require integration to obtain the values of curvature κ and width w , or any other parameters that vary along the track, making the process more complicated.

To switch from time t to curvilinear abscissa s as the independent variable, the dynamic problem must be reformulated. The original system can be represented by a set of first-order Ordinary Differential Equations (ODEs) of the form:

$$\frac{dz}{ds} = \mathbf{f}(z, u) \quad (2.4)$$

where z is the vector of states and u is the vector of inputs. We can replace the time variable t with the arc length s using the following relationship:

$$\frac{dz}{ds} = \mathbf{f}(z, u) \frac{dt}{ds} = \mathbf{f}(z, u) t' \quad (2.5)$$

Here, t' represents time's rate of change with respect to the curvilinear abscissa s . By substituting this into the dynamic model, we can reformulate system's equations accordingly.

This transformation allows for a more straightforward interaction of vehicle's movement with track's geometric characteristics, facilitating problem formulation.

2.2 Model of the vehicle

For the vehicle dynamics description, we opted for a simplified single-track model with rear-wheel drive and front-wheel steering, which is a common configuration for sportcars. This representation provides a straightforward approach to analyze vehicle's motion in a two-dimensional plane using seven primary states, if wheel dynamics are included. To alleviate the problem complexity, we intentionally decided to neglect nonlinear effects such as dynamic load transfer and rolling resistance. The vehicle dynamics can be described by the following equations with respect to curvilinear abscissa s :

$$\frac{dv_x}{ds} = (F_{x,r} + F_{x,f} \cos(\delta) - F_{y,f} \sin(\delta) - F_{drag} + Mv_y\Omega_z) \frac{1}{M} t' \quad (2.6)$$

$$\frac{dv_y}{ds} = (F_{y,r} + F_{y,f} \cos(\delta) + F_{x,f} \sin(\delta) - Mv_x\Omega_z) \frac{1}{M} t' \quad (2.7)$$

$$\frac{d\Omega_z}{ds} = F_{y,f} l_f \cos(\delta) + F_{x,f} l_f \sin(\delta) - F_{y,r} l_r) \frac{1}{I_z} t' \quad (2.8)$$

In these equations, v_x and v_y denote respectively the longitudinal and lateral velocities in the vehicle-fixed frame. The forces $F_{x,f}$ and $F_{y,f}$ correspond to the longitudinal and lateral forces at the front tires relative to their local frame, while $F_{x,r}$ and $F_{y,r}$ represent the longitudinal and lateral forces at the rear tires. The aerodynamic drag force is denoted by F_{drag} , and δ represents the steering angle of the front wheels. Additionally, l_f and l_r describe respectively the longitudinal distance from the center of gravity of the vehicle to the front and rear axles, M is vehicle's mass, and I_z represents vehicle's moment of inertia around its local z-axis.

To capture the dynamics of wheel rotation, we introduce two additional states into the model. These include the angular velocities Ω_f and Ω_r of the front and rear wheels, respectively. The applied torque is denoted by T_i ($i = f, r$), R_w is the wheel radius, and J_w represents the rotational inertia of the wheels:

$$\frac{d\Omega_f}{ds} = (T_f - F_{x,f} R_w) \frac{1}{J_w} t' \quad (2.9)$$

$$\frac{d\Omega_r}{ds} = (T_r - F_{x,r} R_w) \frac{1}{J_w} t' \quad (2.10)$$

2.2.1 Tire Forces

A crucial aspect of vehicle modeling is accurately defining the forces acting on the tires. These forces include the vertical load, the longitudinal force and the lateral force.

Vertical Load

The vertical load is computed considering the static weight distribution on each axle and the aerodynamic downforce F_{lift} .

$$F_{z,f} = Mg \frac{l_r}{l} + \frac{F_{lift}}{2} \quad (2.11)$$

$$F_{z,r} = Mg \frac{l_f}{l} + \frac{F_{lift}}{2} \quad (2.12)$$

Where g is the gravity acceleration and l is the vehicle wheelbase.

Lateral and Longitudinal Tire Forces

The lateral and longitudinal tire forces can be computed using the Pacejka's Magic Formula, which describes the tire slip [1]:

$$F_{x,f} = D_{x,f} \sin(c_x \arctan(b_x s_{x,f} - e_x(b_x s_{x,f} - \arctan(b_x s_{x,f})))) \quad (2.13)$$

$$F_{x,r} = D_{x,r} \sin(c_x \arctan(b_x s_{x,r} - e_x(b_x s_{x,r} - \arctan(b_x s_{x,r})))) \quad (2.14)$$

$$F_{y,f} = D_{y,f} \sin(c_y \arctan(b_y \lambda_f - e_y(b_y \lambda_f - \arctan(b_y \lambda_f)))) \quad (2.15)$$

$$F_{y,r} = D_{y,r} \sin(c_y \arctan(b_y \lambda_r - e_y(b_y \lambda_r - \arctan(b_y \lambda_r)))) \quad (2.16)$$

with

$$D_{x,f} = \mu_x F_{z,f} \quad (2.17)$$

$$D_{x,r} = \mu_x F_{z,r} \quad (2.18)$$

$$D_{y,f} = \mu_y F_{z,f} \quad (2.19)$$

$$D_{y,r} = \mu_y F_{z,r} \quad (2.20)$$

$$\mu_x = \mu_0 + p_{D2} \frac{F_{z,x} - F_{z0}}{F_{z0}} \quad (2.21)$$

$$\mu_y = \mu_0 + p_{D2} \frac{F_{z,y} - F_{z0}}{F_{z0}} \quad (2.22)$$

In this formula, the coefficients c_x , c_y , b_x , b_y , e_x , and e_y are constants. The values $D_{x,r}$, $D_{x,f}$, $D_{y,r}$, and $D_{y,f}$ depend on the vertical load and the variable coefficient of friction. μ_0 represents the coefficient of friction at the nominal load F_{z0} and p_{D2} is a negative coefficient determining the sensitivity of the tire load.

The slip angles are defined as:

$$\lambda_f = \delta - \arctan\left(\frac{\Omega_z l_f + v_y}{v_x}\right) \quad (2.23)$$

$$\lambda_r = \arctan\left(\frac{\Omega_z l_r - v_y}{v_x}\right) \quad (2.24)$$

The longitudinal slip ratios are given by:

$$s_{x,f} = \frac{\Omega_f R_w - v_{x,f}}{v_{x,f}} \quad (2.25)$$

$$s_{x,r} = \frac{\Omega_r R_w - v_{x,r}}{v_{x,r}} \quad (2.26)$$

Where $v_{x,r}$ and $v_{x,f}$ are the longitudinal speeds of the rear and front wheels respectively, in general different from the longitudinal speed (see fig. 2.1) of the vehicle and obtained as:

$$v_{x,f} = \cos(\lambda_f) \sqrt{(v_y \pm \Omega_z l_f)^2 + v_x^2} \quad (2.27)$$

$$v_{x,r} = \cos(\lambda_r) \sqrt{(v_y \pm \Omega_z l_r)^2 + v_x^2} \quad (2.28)$$

The overall behavior of the tires is governed by applying a constraint that encompasses both the total longitudinal and lateral forces. This constraint ensures that the combination of these forces remains within the boundaries defined by the friction ellipse. The friction ellipse represents the maximum grip the tires can provide, and the constraint effectively limits the total forces so that they do not exceed this grip, maintaining control and stability:

$$\mu_f^2 - \frac{F_{x,f}^2 + F_{y,f}^2}{F_{z,f}^2} \geq 0 \quad (2.29)$$

$$\mu_r^2 - \frac{F_{x,r}^2 + F_{y,r}^2}{F_{z,r}^2} \geq 0 \quad (2.30)$$

Aerodynamic Forces

Two primary aerodynamic forces acting on the vehicle are the drag force and the lift force. These can be defined as follows:

$$F_{drag} = \frac{1}{2} \rho_{air} C_d A_f v_x^2 \quad (2.31)$$

$$F_{lift} = \frac{1}{2} \rho_{air} C_l A_f v_x^2 \quad (2.32)$$

In these equations, C_d and C_l represent respectively the drag and lift coefficients, ρ_{air} represents the air density and A_f represents the vehicle front area.

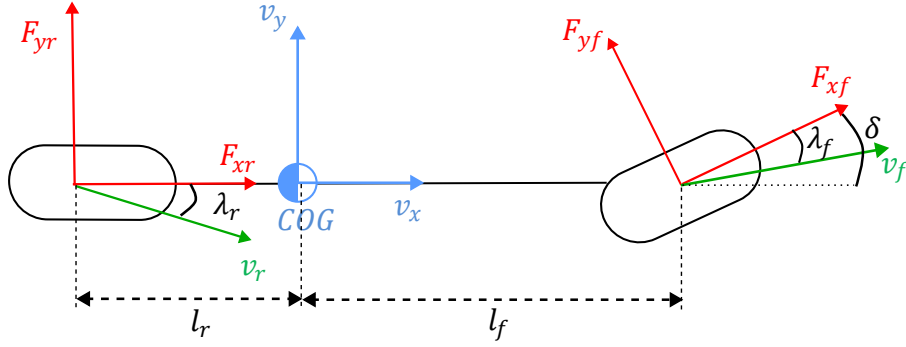


Figure 2.1: Single track model illustration

2.2.2 Input of the system and torque constraints

System's inputs include several critical parameters that govern its behavior. The first input, denoted by δ , represents the steering angle of the front wheel. This angle plays a crucial role in determining the direction and stability of the vehicle as it navigates through different trajectories.

Another essential input is T_{brk} , which signifies the total brake torque applied to the vehicle. This torque is not applied uniformly across all wheels but is instead distributed between the front and rear axles. The distribution is managed by a coefficient K_{brk} , which defines how much of the total brake torque is allocated to the front axle versus the rear axle. This coefficient is crucial for optimizing braking performance and ensuring that the vehicle remains stable during deceleration.

Additionally, the drive torque, denoted as T_{drv} , is an input applied specifically to the rear axle. This torque is responsible for propelling the vehicle forward and is a key factor in controlling acceleration and maintaining speed.

Given these inputs, the torque distributed to the front axle of the vehicle (T_f) and to the rear axle (T_r) are computed as follows:

$$T_f = T_{brk} K_{brk} \quad (2.33)$$

$$T_r = T_{drv} + T_{brk}(1 - K_{brk}) \quad (2.34)$$

We need to include another constraint to impose that braking is not applied at the same time as the driving torque:

$$T_{drv} T_{brk} = 0 \quad (2.35)$$

Chapter 3

Method

We are now outlining the process for solving the Optimal Control Problem. The first crucial step is to transform the OCP into a discrete nonlinear problem. This transformation is essential to apply numerical optimization methods to the originally continuous problem. In our case, we use a method known as *direct orthogonal collocation*. This approach is particularly effective since it allows us to convert the control problem, which is initially formulated in continuous terms, into a discrete one, making it more manageable for numerical solvers. The direct orthogonal collocation method works by discretizing the time domain and approximating the state and control variables using polynomial functions within each time interval.

This technique not only facilitates the solution of the problem but also ensures an accurate representation of system's dynamic behavior while keeping computational complexity at a manageable level. Therefore, converting the OCP into a discrete NLP using *direct orthogonal collocation* is a critical step in the entire optimization process.

3.1 Modeling of the optimization problem

In the context of solving the Minimum Lap Time Problem, the primary goal is to minimize the total time required for a car to complete one full lap around a racing circuit. This task is subject to various constraints, which are linked to both the dynamic behavior and physical limitations of the vehicle, as well as the specific geometry and layout of the circuit itself.

More specifically, these constraints may include aspects such as car's maximum speed, acceleration, tire grip, and braking capabilities, all of which must be carefully balanced to achieve the fastest possible lap time. Additionally, track's features, such as its curvature and surface conditions, also impose limits that must be considered.

In our specific scenario, the objective function is defined as shown below and can be expressed using both time and the variable s as independent variables:

$$J(x) = \int_0^{t_f} 1 dt = \int_0^{s_f} t'(x) ds \quad (3.1)$$

where t_f and s_f denote respectively the final time and final distance and x includes the optimization variables, which generally consist of the control inputs u and the states z when multiple shooting approaches are implemented.

3.2 Discretization of the problem

3.2.1 Transcription

The process of converting the optimal control problem into a discrete Nonlinear Problem is known as *transcription*. In this context, we can apply a technique referred to as *direct orthogonal local collocation*, which involves using barycentric Lagrange interpolation to achieve the desired results.

Direct orthogonal local collocation - theory

By utilizing this approach, the states $z(t)$ can be approximated over the collocation interval $t \in [t_k, t_{k+1}]$, which is a subset of the entire time domain $[t_0, t_f]$. This approximation is accomplished through a polynomial of degree d_{th} , denoted as $p_k(t, v_k)$, where the polynomial is defined to be linear in terms of the coefficients $v_k \in \mathbb{R}^{n_{int}(d_{th}+1)}$. Here, n_{int} represents the number of collocation intervals. These coefficients play a crucial role in shaping the polynomial.

The polynomials $p_k(t, v_k)$ are carefully constructed as *Lagrange polynomials*, which are specifically tailored to provide a highly accurate and reliable representation of the solution within each designated collocation interval. These polynomials play a critical role in ensuring that the approximation closely follows the behavior of the true solution over the specific time intervals. For a given time interval $[t_k, t_{k+1}]$ and a corresponding set of collocation times $t_{k,0}, t_{k,1}, \dots, t_{k,d_{th}}$, the polynomial can be constructed in a straightforward manner. This construction leverages the properties of the Lagrange basis polynomials, which are designed to interpolate the function at these specific collocation points, ensuring that the polynomial not only passes through these points but also adheres to the desired accuracy criteria within the entire interval.

The polynomials $p_k(t, v_k)$ thus provides a robust and precise approximation of the solution across the collocation interval, encapsulating the dynamics of the system effectively.

$$p_k(t, v_k) = \sum_{i=0}^{d_{th}} v_{k,i} l_{k,i}(t) \quad (3.2)$$

where:

$$l_{k,i}(t) = \prod_{j=0, j \neq i}^{d_{th}} \frac{t - t_{k,j}}{t_{k,i} - t_{k,j}} \quad (3.3)$$

where $v_{k,i} \in \mathbb{R}^n$ represents a subset of the polynomial coefficients v_k . This subset $v_{k,i}$ is specifically selected to correspond to the size of the state vector of the system under consideration. Additionally, the collocation times, denoted as $t_{k,j}$, are chosen to lie within the interval $[t_k, t_{k+1}]$. These collocation times are critical for accurately capturing the behavior of the system within each interval, ensuring that the polynomial approximates the solution effectively over the specified time range.

The polynomials $l_{k,i}$ have by construction the property:

$$l_{k,i} = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases} \quad (3.4)$$

This property indicates that the interpolation polynomial $p_k(t_{k,i}, v_k)$ is specifically designed to pass exactly through the set of interpolation points $v_{k,i}$. In other words, for each interpolation point $t_{k,i}$, the corresponding value of the polynomial $p_k(t_{k,i})$ matches precisely with the given value $v_{k,i}$.

This ensures that the polynomial aligns perfectly with the data points provided, thereby accurately representing the underlying function at these specific points.

$$p_k(t_{k,i}, v_k) = v_{k,i} \quad (3.5)$$

that holds for $i = 0, \dots, d_{th}$

Furthermore, the basis polynomials $l_{k,i}$ are characterized by the important property of orthogonality. This orthogonality implies that each basis polynomial $l_{k,i}$ is orthogonal to every other basis polynomial in the set, which means that the inner product of any two distinct basis polynomials is zero.

$$\int_{l_k}^{l_{k+1}} l_{k,i}(t) l_{k,j}(t) dt = 0, i \neq j \quad (3.6)$$

To carry out the integration over the time interval $[t_k, t_{k+1}]$ using the polynomial $p_k(t, v_k)$, it is essential to carefully choose appropriate collocation variables $v_k \in \mathbb{R}^{n_{int}(d+1)}$. The selection of these variables is crucial because the integration process involves solving a system of algebraic equations. These equations are designed to ensure that the polynomial $p_k(t, v_k)$ serves as a highly accurate representation of the state trajectories over the entire time interval. By solving these equations, we effectively guarantee that the polynomial captures the dynamics of the system with a high degree of precision.

Consider the scenario where we have an initial state value s_k at the starting time t_k . In this context, the collocation equations play a pivotal role, particularly for a simple ordinary differential equation (ODE) of the form $\dot{z}(t) = \mathbf{f}(z(t), t)$. These collocation equations impose a set of $n_{int}(d_{th} + 1)$ conditions, which must be satisfied to ensure the correct evolution of the state variables:

- $p_k(t_k, v_k) = s_k$
The polynomial $p_k(t_k, v_k)$ is required to satisfy the initial conditions at the specific time $t_k = t_{k,0}$, which corresponds to the start of the interval under consideration. This condition ensures that the polynomial accurately represents the initial state of the system. Observing the behavior of the polynomial, we note that at the initial time $t_k = t_{k,0}$, we have $p_k(t_k, v_k) = v_{k,0}$. To satisfy the initial conditions, we must therefore equate this initial value $v_{k,0}$ to the given parameter s_k , ensuring that the polynomial $p_k(t_k, v_k)$ adheres to the necessary constraints at the beginning of the interval. Consequently, by setting $v_{k,0} = s_k$, the initial conditions are fully satisfied, and the polynomial correctly reflects the required initial state.
- All the $p_k(t_{k,i}, v_k)$ must adhere to the model dynamics not only at the initial time $t_{k,0}$ but also at each of the remaining collocation times within the interval, specifically at $t_{k,1}, t_{k,2}, \dots, t_{k,d_{th}}$. These collocation points are crucial, as they represent the discrete moments in time at which the polynomial solution is evaluated to ensure that it accurately follows the underlying dynamics of the model. For each of these collocation times $t_{k,1}, t_{k,2}, \dots, t_{k,d_{th}}$, the polynomial must satisfy the governing equations that describe system's behavior.

$$\dot{p}_k(t_{k,i}, v_k) = \mathbf{f}(p_k(t_{k,i}, v_k), t_{k,i}) \quad (3.7)$$

The integration of the dynamic of the system over the time interval $[t_k, t_{k+1}]$ is performed via solving the collocation equations:

$$c_k(v_k, t_{k,i}, s_k) = \begin{pmatrix} v_{k,0} - s_k \\ \dot{p}_k(t_{k,1}, v_k) - \mathbf{f}(v_{k,1}, t_k) \\ \vdots \\ \dot{p}_k(t_{k,d_{th}}, v_k) - \mathbf{f}(v_{k,d_{th}}, t_{k,i}) \end{pmatrix} = 0 \quad (3.8)$$

But now, the key issue that arises is how to effectively select the collocation points. It is crucial to emphasize that making a well-informed and appropriate selection of collocation points is fundamental, as it directly influences the accuracy and can significantly enhance the order of integration, potentially achieving a very high degree of precision.

To gain a deeper understanding of this concept, we can turn our attention to the principle of *Gauss quadrature*. This principle plays a vital role in numerical integration, particularly when dealing with polynomials. Specifically, if we assume that the function in question, denoted as $p_k(t, v_k)$, is a polynomial of order $2d_{th}$, the Gaussian quadrature formula becomes highly relevant. It provides, for any given value of v_k , the exact equality expressed as follows:

$$\int_{t_k}^{t_{k+1}} \dot{p}_k(t, v_k) dt = (t_{k+1} - t_k) \sum_{i=1}^m \omega_i \dot{p}_k(t_{k,i}, v_k) \quad (3.9)$$

for a given choice of the value of ω_i and of the collocation points $t_{k,i}$.

The process of determining the collocation time is achieved by carefully selecting the collocation points $t_{k,i}$ such that they correspond to the zeros of the orthogonal Legendre polynomials within the specific interval $[t_k, t_{k+1}]$. This method ensures that the collocation points are optimally distributed within the interval, leveraging the properties of these polynomials to enhance the accuracy of the approximation.

This strategic selection of collocation times is a well-known technique in numerical analysis and is referred to as *Gauss-Legendre collocation*. The name reflects the use of Gauss quadrature in conjunction with Legendre polynomials, which are particularly suited for this purpose due to their orthogonality and the resulting optimal distribution of points. This approach is widely recognized for its effectiveness in achieving high precision in various computational methods.

Considering the specific interval $[0, 1]$, the collocation points $t_{k,i}$ for $i = 1, \dots, 4$ are provided in the table below [3.1] and are the ones that we used in the definition of the matrixes in the matlab code:

d_{th}	Gauss-Legendre Collocation
1	0.5000
2	0.21132 0.78867
3	0.11270 0.500 0.88729
4	0.06943 0.3300 0.66999 0.93056

Table 3.1. Collocation times $t_{k,1} \dots t_{k,d_{th}}$ for $d_{th} = 1, \dots, 4$ in the interval $[0, 1]$

Notice that the points $t_{k,1}, \dots, t_{k,d_{th}}$ are inside the collocation interval and are symmetric around the midpoints.

Let's note that the exact state trajectory $z(t)$ of the ODE $\dot{z}(t) = \mathbf{f}(z(t), t)$ fulfills the following equations.

$$z(t_{k+1}) = z(t_k) + \int_{t_k}^{t_{k+1}} \mathbf{f}(z(t), t) dt \quad (3.10)$$

Let's assume that the trajectory $z(t)$ can be exactly captured by the polynomial $p_k(t, v_k)$ of degree $2d_{th}$ on the time interval $[t_k, t_{k+1}]$; finally we can obtain

$$p_k(t_{k+1}, v_k) = p_k(t_k, v_k) + (t_{k+1} - t_k) \sum_{i=1}^{d_{th}} \omega_i \mathbf{f}(p_k(t_{k,i}, v_k), t_{k,i}) \quad (3.11)$$

such that $s_{k+1} = p_k(t_{k+1}, v_k) = z(t_{k+1})$ holds.

Approach applied to the MLTP problem

The entire length of the track, denoted as N , can be subdivided into N finite elements. By doing so, we can define $N + 1$ grid points, which we shall refer to as s_K , where each element has an arbitrary length ds_k . As we discussed in the previous section, within each interval, the states are approximated using a Legendre polynomial of a specified degree d_{th} . For this purpose, we have the freedom to choose collocation points $s_{k,j}$, which are determined as the roots of the Legendre polynomials.

It is important to note that these roots are typically defined within the interval $[-1, 1]$. However, they are not strictly confined to this interval. By applying an appropriate scaling transformation, these collocation points can be adapted to any other desired interval. This flexibility allows us to adjust the interval to better suit the specific problem at hand, ensuring that the grid points are appropriately spaced for accurate approximation of the states.

We can define $\dot{\mathbf{L}}_k(s_{k,j})$ as the derivative of the polynomial at the collocation point.

Let's define $\mathbf{L}_k = [l_k(s_{k,1}), l_k(s_{k,2}), \dots, l_k(s_{k,d_{th}})]$, where $l_k(s)$ are the Lagrange polynomial of order d_{th} in the interval $[s_k, s_{k+1}]$ and $s_{k,j}$ are the collocation points.

Notice that $l_k(s_{k,j}) = X_{k,j}$. Then, the derivatives at the collocation points can be found as:

$$\dot{\mathbf{L}}_k = \frac{C_{col}}{ds_k} \mathbf{L}_k \quad (3.12)$$

Similarly, the contribution of each finite element to the objective function can be computed. This process involves evaluating the objective function at specific collocation points within each finite element. These contributions are collectively represented by the vector $\mathbf{S}_{f_k} = [S_f(\mathbf{s}_{k,1}), S_f(\mathbf{s}_{k,2}), \dots, S_f(\mathbf{s}_{k,d})]$, where each term corresponds to the value of the objective function at the respective collocation point $\mathbf{s}_{k,j}$. Additionally, this approach allows us to extrapolate the state variables at the end of each interval, ensuring that the behavior of the system can be accurately predicted throughout the entire domain.

The matrices C_{col} , B_{col} , and D_{col} can be computed specifically for the interval $[0, 1]$, which serves as the standard reference interval in this context. The structure and values of these matrices are not fixed but rather are influenced by the degree of the polynomial that has been selected for the approximation.

$$J_k = B_{col} \mathbf{S}_{f_k} d_{s_k} \quad (3.13)$$

$$Z_{end} = D_{col} [z_k, z_{k,1}, \dots, z_{k,d}] \quad (3.14)$$

In our specific problem the corresponding matrixes $C_{col} \in R^{(d_{th}+1) \times d_{th}}$, $D_{col} \in R^{d_{th}+1}$ and $B_{col} \in R^{d_{th}}$ for the interval $[0, 1]$ can be found by mean of *Casadi* function *collocation_coeff*($\cdot$), passing as input to the function the collocation coefficients in table (3.1). Matrices from (3.15) to (3.17) contain the results for a Gauss-Legendre Collocation polynomial in $[0, 1]$ with $d_{th} = 3$.

$$C_{col} = \begin{bmatrix} -6 & 3 & -6 \\ 5 & -5.72749 & 10.164 \\ 1.16398 & 2 & -9.16398 \\ -0.163978 & 0.727486 & 5 \end{bmatrix} \quad (3.15)$$

$$D_{col} = \begin{bmatrix} -1 \\ 1.66667 \\ -1.33333 \\ 1.66667 \end{bmatrix} \quad (3.16)$$

$$B_{col} = \begin{bmatrix} 0.277778 \\ 0.444444 \\ 0.277778 \end{bmatrix} \quad (3.17)$$

The collocation constraints are formulated by aligning the derivatives of the polynomials with the true dynamics of the system at the designated collocation points. Additionally, these constraints ensure continuity by matching the extrapolated final state of each finite element with the initial state of the subsequent element.

Finally we can formalize the NLP formulation as follow:

$$\underset{z_k, z_{k,j}, u_k}{\text{minimize}} \quad {}^*J = \sum_{k=1}^{k=N} B_{col} \mathbf{S}_{f_k} ds_k \quad (3.18)$$

$$\text{subject to} \quad \frac{d}{ds} l_k(s_{k,j}) - \mathbf{f}(z_{k,j}, u_k, \kappa_k) = 0 \quad \forall k \in [0, N] \quad \text{and} \quad j \in [1, d] \quad (3.19)$$

$$z_{end} - z_k = 0 \quad \forall k \in [0, N+1] \quad (3.20)$$

$$\mathbf{H}_{lb} \leq \mathbf{H}_{ub} \quad \forall k \in [0, N] \quad (3.21)$$

$$B_{lb,k} \leq h(z_k, u_k, \kappa_k) \leq B_{ub,k} \quad \text{for} \quad k = 0, \dots, k = N+1 \quad (3.22)$$

* To help the solver, our optimization problem uses J^2 in place of J .

In our problem the decision variables become:

- states z_k
- *helper* states $z_{k,j}$
- inputs u_k

All the decision variables are bounded to limit the minimum and maximum values.

3.3 Scaling

Another essential step in optimizing the solver performances is the scaling of the decision variables, as this can have a substantial impact on the accuracy and efficiency of the results. Scaling helps to normalize the range of the decision variables, making them more manageable for the solver to process. This procedure is relatively simple, given that the maximum value of the decision variables is already known. By dividing each decision variable by its corresponding maximum value (in absolute value), we standardize the variables, ensuring they all fall within a similar range.

Chapter 4

Theoretical Background

As previously discussed the MLTP problem described as an Optimal Control Problem was transformed into a discrete NLP using direct orthogonal collocation, also called transcription, after scaling the optimization variables.

The optimization problem for the solver subroutine takes the following form:

$$\begin{aligned} & \min_{x \in \mathbb{R}^n} f(x) \\ & \text{subject to:} \\ & \quad Ax = b \\ & \quad Cx \geq d \\ & \quad g(x) = 0 \\ & \quad h(x) \geq 0 \end{aligned} \tag{4.1}$$

with $f, g, h \in C^2$, $f : \mathbb{R}^n \rightarrow \mathbb{R}$, $g : \mathbb{R}^n \rightarrow \mathbb{R}^p$, $h : \mathbb{R}^n \rightarrow \mathbb{R}^q$.

Before delving into the choices made to achieve the most efficient solution possible, we will briefly cover the theory behind the optimization routine, implemented through a "myfmincon" function.

We must include this discussion to ensure the reasons behind our choices are clear.

4.1 Theory behind the solver

Considering (4.1), we shall first recall the definition of feasible set Ω :

$$\Omega = \{x \in \mathbb{R}^n : g(x) = 0, h(x) \geq 0\} \tag{4.2}$$

To state sensible optimality conditions for a minimizer x^* , we shall analyze the features of the optimization problem at the border of Ω .

Now, consider a point x^* lying on the boundary of Ω . From such a point, not all search directions are feasible. The feasible directions must in fact point to the interior or along the boundary of Ω . These are called tangent vectors to Ω .

Definition of tangent vector

A vector $\mathbf{p} \in \mathbb{R}^n$ is a tangent vector to Ω at point $x^* \in \Omega$, if $\exists$ a smooth curve $\bar{x}(t)$, $\bar{x} : [0, \bar{\epsilon}) \rightarrow \Omega$, such that $\bar{x}(0) = x^*$ and $\mathbf{p} = \frac{d\bar{x}(0)}{dt}$.

Definition tangent cone

The tangent cone $T_\Omega(x^*)$ to Ω at point $x^* \in \Omega$ is

$$T_\Omega(x^*) = \{\mathbf{p} : \mathbf{p} \text{ is a tangent vector to } \Omega \text{ at } x^*\}.$$

Definition active and inactive constraints

A constraint $h_i(x) \geq 0$ is *active* at x^* if $h_i(x^*) = 0$. Otherwise, it is *inactive*.

If $x^* \in \Omega$, equality constraints are always active by definition. We can now also define the active set.

Definition active set

The active set $A(x^*)$ is the set of indexes of all active constraints at point x^* .

Thus, for a feasible point $x^* \in \Omega$, the active set will include all the indexes j of functions $g_j(x)$, $j = 1, \dots, p$, plus possibly the indexes of active inequality constraints $h_i(x)$ such that $h_i(x^*) = 0$, $i = 1, \dots, q$.

Definition of Linearly Independent Constraint Qualification - LICQ

LICQ holds if all the gradients of $h_i(x)$, $\forall i \in A(x^*)$, and of $g(x^*)$ are linearly independent, i.e., these vectors define a subspace of $\mathbb{R}^n$.

All these definition eventually lead to the following necessary optimality condition.

Necessary optimality condition of the first order for constrained problems

Theorem 4.1.1. *If x^* is a local minimizer of (4.1) and LICQ holds at x^* , then:*

1. $x^* \in \Omega$;
2. For all $\mathbf{p} \in \mathbb{R}^n$: if $\nabla_x g(x^*)^T \mathbf{p} = 0$ and $\nabla_x h_i(x^*)^T \mathbf{p} \geq 0$ for all $i \in A(x^*)$, then $\mathbf{p}^T \nabla_x f(x^*) \geq 0$.

The theorem enunciated above is still not immediately useful for designing a solution algorithm, as one would still need to check all possible directions $\mathbf{p} \in T_\Omega(x^*)$, so we need to reformulate them into one more usable. After some mathematical steps, it is possible to achieve the following result.

Necessary optimality condition of the first order for constrained problems: KKT conditions

Theorem 4.1.2. *If x^* is a local minimizer of (4.1) and LICQ holds at x^* , then there exist Lagrange multipliers $\lambda^* \in \mathbb{R}^p$ and $\mu^* \in \mathbb{R}^q$ such that:*

$$\nabla_x L(x^*, \lambda^*, \mu^*) = \nabla_x f(x^*) - \nabla_x g(x^*) \lambda^* - \nabla_x h(x^*) \mu^* = 0 \quad (4.3)$$

$$g(x^*) = 0 \quad (4.4)$$

$$h(x^*) \geq 0 \quad (4.5)$$

$$\mu^* \geq 0 \quad (4.6)$$

$$h(x^*)^T \mu^* = 0 \quad (4.7)$$

In (4.3), $L(\cdot)$ is the Lagrangian function. We name a triplet (x^*, λ^*, μ^*) satisfying the theorem a “KKT triplet”. This theorem alone does not allow to distinguish between local minimizers and maximizers, and thus second order optimality condition are also needed for the optimization algorithm to

work. Let's first state an instrumental definition.

Definition cone of critical directions

The cone of critical directions $C(x^*, \mu^*)$ at the KKT point (x^*, λ^*, μ^*) is:

$$C(x^*, \mu^*) = \{\mathbf{p} : \nabla_x g(x^*)^T \mathbf{p} = 0, \nabla_x h_i(x^*)^T \mathbf{p} \geq 0, \forall i \in A(x^*), \nabla_x h_i(x^*)^T \mathbf{p} = 0, \forall i \in A(x^*) \text{ such that } \mu_i^* > 0\}.$$

Necessary optimality condition of the second order for constrained problems

Theorem 4.1.3. *If x^* is a local minimizer of (4.1) and LICQ holds at x^* , then for all $\mathbf{p} \in C(x^*, \mu^*)$, it holds that*

$$\mathbf{p}^T \nabla_x^2 L(x^*, \lambda^*, \mu^*) \mathbf{p} \geq 0.$$

Sufficient optimality condition of the second order for constrained problems

Theorem 4.1.4. *If the triplet (x^*, λ^*, μ^*) satisfies the KKT conditions, if LICQ holds at x^* , and if for all $\mathbf{p} \in C(x^*, \mu^*)$ it holds that*

$$\mathbf{p}^T \nabla_x^2 L(x^*, \lambda^*, \mu^*) \mathbf{p} > 0,$$

then x^ is a local minimizer of (4.1).*

After having characterized the necessary and sufficient optimality conditions for constrained continuous NLPs, we now focus on solution algorithms. As a preliminary step, an overview of Quadratic Programming (QP) approaches is given in this section. Although (convex) QP solvers are today extremely efficient, so that one can use one of the many existing toolboxes and software suites, it is still important to understand their working principles. Let us recall a standard form of a QP:

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & c^T x + \frac{1}{2} x^T H x \\ \text{subject to:} \quad & \\ & Ax + b = 0 \\ & Cx + d \geq 0 \end{aligned} \tag{4.8}$$

where $A \in \mathbb{R}^{p \times n}$, $b \in \mathbb{R}^p$, $C \in \mathbb{R}^{q \times n}$, $d \in \mathbb{R}^q$. The QP is convex if the Hessian $H \geq 0$.

In this case, the Lagrangian reads:

$$L(x, \lambda, \mu) = c^T x + \frac{1}{2} x^T H x - (Ax + b)^T \lambda - (Cx + d)^T \mu$$

and the KKT conditions are:

$$c + Hx^* - A^T \lambda^* - C^T \mu^* = 0 \tag{4.9}$$

$$Ax^* + b = 0 \tag{4.10}$$

$$Cx^* + d \geq 0 \tag{4.11}$$

$$\mu^* \geq 0 \tag{4.12}$$

$$(Cx^* + d)^T \mu^* = 0 \tag{4.13}$$

If the QP is convex, then a valid KKT triplet yields a global minimizer x^* . Finding a KKT triplet that satisfies equation from (4.9) to (4.13) is a nonlinear and non-smooth problem, due to the complementarity conditions (4.11)-(4.13). There are two main families of methods to deal with this issue:

active-set and interior-point. Both are iterative algorithms that require a feasible initial point x^o . In the practical part we will explain as we reasoned to find such feasible initial point x^o . Since the aforementioned *myfmincon* uses the interior-point method to solve QP-subproblems, the discussion on the active-set method will not be reported here. The idea of interior-point strategies is to replace the non-smooth complementarity conditions (4.11)-(4.13) with a smooth nonlinear equation, which is gradually changed until the original conditions are recovered:

$$\begin{aligned} c + Hx^* - A^T\lambda^* - C^T\mu^* &= 0 \\ Ax^* + b &= 0 \\ (C_i x^* + d_i)\mu_i^* - \tau &= 0, \quad \forall i = 1, \dots, q \end{aligned} \quad (4.14)$$

where C_i is the i -th row of matrix C , and $\tau > 0$ is a scalar tuned by the iterative algorithm. In this way, for a fixed τ one needs to solve a set of $(n + p + q)$ nonlinear equations in $(n + p + q)$ unknowns. This can be done, e.g., with Newton's method and yields a triplet $(x^*(\tau), \lambda^*(\tau), \mu^*(\tau))$. Then, the value of τ is decreased, and equations (4.14) are solved again by initializing from the previous solution. This approach generates a sequence $\{(x^*(\tau), \lambda^*(\tau), \mu^*(\tau))\}$ that converges to a KKT triplet (x^*, λ^*, μ^*) as $\tau \rightarrow 0$. Such a sequence is called the "central path".

Sequential Quadratic Programming algorithms are iterative solvers for general NLPs of the form (4.1). They attempt to generate a sequence $\{(x_k, \lambda_k, \mu_k)\}$ of primal and dual variables, i.e., x and the multipliers λ, μ , that converges to a KKT triplet corresponding to a local minimizer. The main idea is to approximate locally, around each value (x_k, λ_k, μ_k) , the nonlinear program as a QP, i.e., the cost function f is approximated by a quadratic one, and the constraints are linearized. The solution to such a QP yields search directions for both the primal and dual variables, i.e., x and the multipliers λ, μ . Then, a line search sub-routine is carried out to compute the next iterate, $(x_{k+1}, \lambda_{k+1}, \mu_{k+1})$, when a new QP is derived and solved, and so on until a convergence criterion is met. In our work, we decided to employ the Broyden-Fletcher-Goldfarb-Shanno (BFGS) update formula for the Hessian estimation.

Constrained BFGS

Consider the first-order Taylor expansion of $\nabla_x L(x, \lambda_{k+1}, \mu_{k+1})$ evaluated at $x = x_{k+1}$:

$$\nabla_x L(x_{k+1}, \lambda_{k+1}, \mu_{k+1}) = \nabla_x L(x_k, \lambda_{k+1}, \mu_{k+1}) + \nabla_x^2 L(x_k, \lambda_{k+1}, \mu_{k+1})(x_{k+1} - x_k).$$

Then, defining:

$$\bar{s} = x_{k+1} - x_k, \quad y = \nabla_x L(x_{k+1}, \lambda_{k+1}, \mu_{k+1}) - \nabla_x L(x_k, \lambda_{k+1}, \mu_{k+1}),$$

the problem is to find H_{k+1} such that:

$$y = H_{k+1} \bar{s}.$$

A possible solution is to compute H_{k+1} as a function of H_k , $\bar{s}$, and y with the BFGS update:

$$H_{k+1} = H_k - \frac{H_k \bar{s} \bar{s}^T H_k}{\bar{s}^T H_k \bar{s}} + \frac{y y^T}{\bar{s}^T y},$$

and use Powell's trick to ensure positive curvature along $\bar{s}$, i.e., replacing $\tilde{y}$ in the formula above instead of y , where:

$$\tilde{y} = \begin{cases} y & \text{if } y^T \bar{s} > \gamma \bar{s}^T H_k \bar{s}, \\ y + \frac{\gamma \bar{s}^T H_k \bar{s} - \bar{s}^T y}{\bar{s}^T H_k \bar{s} - \bar{s}^T y} (H_k \bar{s} - y) & \text{if } y^T \bar{s} \leq \gamma \bar{s}^T H_k \bar{s}, \end{cases} \quad (4.15)$$

and $\gamma \in (0, 1)$ is a tuning parameter.

BFGS has the advantages of providing a positive definite Hessian estimate, $H_k > 0$, and it doesn't require any additional derivation, since $\nabla_x L(x_k, \lambda_k, \mu_k) = \nabla_x f(x_k) - \nabla_x g(x_k)\lambda_k - \nabla_x h(x_k)\mu_k$, and the gradients $\nabla_x f(x_k)$, $\nabla_x g(x_k)$, $\nabla_x h(x_k)$ have to be computed anyway to obtain the cost and constraints of the QP. So with this method, we obtain a QP sub-problem of the form (4.8). By solving the SQP sub-problem, we obtain search directions $\mathbf{p}_k^*$, $\Delta\lambda_k$, $\Delta\mu_k$, respectively for the primal optimization variables x and the multipliers λ , μ . We now describe how to compute the step length $\bar{t}_k$ such that the overall SQP algorithm guarantees global convergence. In the unconstrained case, this was established by resorting to the Armijo sufficient decrease condition. Here, we need to find a step size such that we improve the cost function value while also satisfying constraints. This feature is achieved by introducing an ℓ_1 -norm merit function:

$$T_1(x) = f(x) + \sum_{i=1}^p \sigma_i |g_i(x)| + \sum_{i=1}^q \tau_i \max(0, -h_i(x)) \quad (4.16)$$

with $\sigma_i, \tau_i > 0$ tunable parameters (as we will see, these are automatically adjusted during the SQP iterations). Note that the terms $\sum_{i=1}^p \sigma_i |g_i(x)|$ and $\sum_{i=1}^q \tau_i \max(0, -h_i(x))$ correspond to weighted ℓ_1 -norms of the constraint violations. For inequality constraints, the lower saturation to zero ensures that these terms contribute to the merit function only when $h_i(x) < 0$. The following result holds.

Theorem 4.1.5. *If x^* is a local minimizer of (4.1) with KKT triplet (x^*, λ^*, μ^*) , and if $\sigma_i > |\lambda_i|$ for $i = 1, \dots, p$ and $\tau_i > |\mu_i|$ for $i = 1, \dots, q$, then x^* is also a local minimizer of $\min_{x \in \mathbb{R}^n} T_1(x)$ (see (4.16)).*

The importance of this theorem stems from the fact that $\min_{x \in \mathbb{R}^n} T_1(x)$ is an unconstrained problem, which shares the same minimizer x^* of the constrained NLP, subject to conditions on the tunable scalars σ_i, τ_i . On the other hand, function $T_1(x)$ is not differentiable, hence the back-tracking line-search strategy to obtain global convergence appears to be not applicable directly. However, the directional derivative of $T_1(x)$, $D(T_1(x), p)$, is well-defined:

$$D(T_1(x), p) = \lim_{\bar{\epsilon} \rightarrow 0^+} \frac{T_1(x + \bar{\epsilon}\mathbf{p}) - T_1(x)}{\bar{\epsilon}\|\mathbf{p}\|} \quad (4.17)$$

This corresponds to the derivative of $T_1(x)$ considering a positive step along direction $\mathbf{p}$. Given $\mathbf{p}_k^*$ from the solution of the QP at the k -th SQP iteration, the directional derivative can be computed in closed form, as stated by the following result.

Theorem 4.1.6. *If $(\mathbf{p}_k^*, \tilde{\lambda}^*, \tilde{\mu}^*)$ is the solution to the k -th SQP iteration with Hessian H_k and gradient*

$\nabla_x f(x_k)$, then it holds:

$$\begin{aligned}
D(T_1(x_k), \mathbf{p}_k^*) &= \nabla_x f(x_k)^T \mathbf{p}_k^* \\
&+ \sum_{i: g_i(x_k) > 0} \sigma_i \nabla_x g_i(x_k)^T \mathbf{p}_k^* \\
&- \sum_{i: g_i(x_k) < 0} \sigma_i \nabla_x g_i(x_k)^T \mathbf{p}_k^* \\
&+ \sum_{i: g_i(x_k) = 0} \sigma_i |\nabla_x g_i(x_k)^T \mathbf{p}_k^*| \\
&- \sum_{i: h_i(x_k) < 0} \tau_i \nabla_x h_i(x_k)^T \mathbf{p}_k^* \\
&+ \sum_{i: h_i(x_k) = 0} \tau_i \max(0, -\nabla_x h_i(x_k)^T \mathbf{p}_k^*)
\end{aligned} \tag{4.18}$$

$D(T_1(x), p)$ is the equivalent of the quantity $\nabla_x f(x_k)^T \mathbf{p}_k$ for unconstrained problems. We need to simplify the equation (4.18) and check whether, at the point $(\mathbf{p}_k^*, \tilde{\lambda}_k^*, \tilde{\mu}_k^*)$ computed in the SQP iteration, $D(T_1(x_k), \mathbf{p}_k^*) \leq 0$, i.e. the directional derivative is a descent direction (or at most equal to zero) for the unconstrained problem $\min_{x \in \mathbb{R}^n} T_1(x)$. To achieve this, we need to use theorems (4.1.5) and (4.1.6), along with equation (4.17), and perform the mathematical steps presented in the theory. We discover that to ensure the global convergence of the SQP algorithm, we must employ the backtracking line search algorithm with the Armijo condition as the stopping criterion, which is given by:

$$T_1(x_k + \bar{t}_k \mathbf{p}_k^*) \leq T_1(x_k) + \bar{t}_k c D(T_1(x_k), \mathbf{p}_k^*), \tag{4.19}$$

where $\bar{t}_k$ is the step size, $\mathbf{p}_k^*$ is the search direction, c is a constant in $(0, 1)$, and $D(T_1(x_k), \mathbf{p}_k^*)$ is the directional derivative.

4.2 Solver choice

In light of the theory discussed, this section details how our algorithm works.

4.2.1 SQP algorithm - using BFGS

Inputs: Cost function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, equality constraints $g : \mathbb{R}^n \rightarrow \mathbb{R}^p$, inequality constraints $h : \mathbb{R}^n \rightarrow \mathbb{R}^q$, initial guess $x_0 \in \mathbb{R}^n$, initial values of σ_i, τ_j for the ℓ_1 -norm merit function, $i = 1, \dots, p$, $j = 1, \dots, q$.

Parameters: Termination tolerances $TOL_\nabla, TOL_x, TOL_f \in (0, 1)$, $TOL_{\text{constr}} > 0$, maximum number of iterations $N_{\text{max}} \in \mathbb{N}$; line-search sub-routine parameters, additional BFGS parameters.

Initialization: $\Delta x_0 = 1$, $\Delta f_0 = 1$, $\lambda_0 = 0$, $\mu_0 = 0$.

Algorithm:

1. Compute $\nabla_x f(x_0)$, $\nabla_x g(x_0)$, $\nabla_x h(x_0)$ with a differentiation method of choice.
2. Check the terminating conditions:

- (a) If $|\nabla_x L(x_k, \lambda_k, \mu_k)^T \mathbf{p}_k| \leq TOL_\nabla$ or $\Delta x_k \leq TOL_x$ or $\Delta f_k \leq TOL_f$ or $k \geq N_{\text{max}}$ and $\|g(x_k)\|_\infty \leq TOL_{\text{constr}}$ and $\min(h(x_k)) \geq TOL_{\text{constr}}$, then exit. In this case, either $x^* = x_k$

is a possible local minimizer (and $f(x^*)$ a local minimum), or the algorithm has ended prematurely but still returns a feasible point.

- (b) If $k \geq N_{\max}$ and $(\|g(x_k)\|_\infty > TOL_{\text{constr}}$ or $\min h(x_k) < TOL_{\text{constr}}$), then exit. In this case, the algorithm has ended prematurely at an unfeasible point, or possibly the problem is not feasible.

3. Solve the QP (4.8):

$$\min_{\mathbf{p}_k} \nabla_x f(x_k)^T \mathbf{p}_k + \frac{1}{2} \mathbf{p}_k^T H_k \mathbf{p}_k \quad \text{s.t.} \quad \nabla_x g(x_k)^T \mathbf{p}_k + g(x_k) = 0, \quad \nabla_x h(x_k)^T \mathbf{p}_k + h(x_k) \geq 0$$

Denote the obtained solution and Lagrange multipliers as $\mathbf{p}_k^*$, $\tilde{\lambda}_k^*$, $\tilde{\mu}_k^*$, and compute:

$$\Delta \lambda_k = (\tilde{\lambda}_k^* - \lambda_k), \quad \Delta \mu_k = (\tilde{\mu}_k^* - \mu_k)$$

with $\lambda_0, \mu_0 = 0$.

4. Update the weights σ_i, τ_i for the merit function $T_1(x)$ as:

$$\sigma_i = \max \left(|\tilde{\lambda}_k^*|, \frac{\sigma_i^- + |\tilde{\lambda}_k^*|}{2} \right), \quad \tau_j = \max \left(|\tilde{\mu}_k^*|, \frac{\tau_j^- + |\tilde{\mu}_k^*|}{2} \right)$$

for all $i = 1, \dots, p$ and all $j = 1, \dots, q$, where σ_i^-, τ_j^- are the previous values.

5. Compute the directional derivative $D(T_1(x_k), \mathbf{p}_k^*)$ as in (4.17), and carry out the backtracking line search (equation 4.19) using the merit function $T_1(x)$ (4.16) and directional derivative $D(T_1(x_k), \mathbf{p}_k^*)$ to compute $\bar{t}_k$. Then update:

$$x_{k+1} = x_k + \bar{t}_k \mathbf{p}_k^*, \quad \lambda_{k+1} = \lambda_k + \bar{t}_k \Delta \lambda_k, \quad \mu_{k+1} = \mu_k + \bar{t}_k \Delta \mu_k$$

6. BFGS update formula :

(a) Compute the gradients $\nabla_x f(x_{k+1}), \nabla_x g(x_{k+1}), \nabla_x h(x_{k+1})$ and:

$$\nabla_x L(x_{k+1}, \lambda_{k+1}, \mu_{k+1}) = \nabla_x f(x_{k+1}) - \nabla_x g(x_{k+1}) \lambda_{k+1} - \nabla_x h(x_{k+1}) \mu_{k+1}$$

$$\nabla_x L(x_k, \lambda_{k+1}, \mu_{k+1}) = \nabla_x f(x_k) - \nabla_x g(x_k) \lambda_{k+1} - \nabla_x h(x_k) \mu_{k+1}$$

Then compute:

$$y = \nabla_x L(x_{k+1}, \lambda_{k+1}, \mu_{k+1}) - \nabla_x L(x_k, \lambda_{k+1}, \mu_{k+1}), \quad \bar{s} = x_{k+1} - x_k$$

(b) If $y^T \bar{s} \leq \gamma \bar{s}^T H_k \bar{s}$ (where $\gamma \in (0, 1)$), then compute $\tilde{y} = y + \frac{\gamma \bar{s}^T H_k \bar{s} - \bar{s}^T y}{\bar{s}^T H_k \bar{s} - \bar{s}^T y} (H_k \bar{s} - y)$.

(c) Compute $H_{k+1} = H_k - \frac{H_k \bar{s} \bar{s}^T H_k}{\bar{s}^T H_k \bar{s}} + \frac{y y^T}{\bar{s}^T y}$ (or using $\tilde{y}$ instead of y).

7. Update the relative changes of optimization variables and cost function, $\Delta x_k, \Delta f_k$, as:

$$\Delta x_k = \frac{\|x_{k+1} - x_k\|}{\max(\bar{\epsilon}, \|x_k\|)}, \quad \Delta f_k = \frac{|f(x_{k+1}) - f(x_k)|}{\max(\bar{\epsilon}, |f(x_k)|)}$$

where $\bar{\epsilon}$ is a small quantity (e.g., 10^{-16}).

8. Set $k = k + 1$, go to step 2.

Chapter 5

Results

In this chapter, we provide an analysis of the obtained results, which were carried out in two distinct phases. Firstly, we focused on simulations involving a single curve, aiming to assess and demonstrate the robustness of the algorithm.

We then expanded our simulation to the entire Yas Marina circuit, allowing us to evaluate algorithm's performance in a more complex and challenging environment.

5.0.1 Simple curve

In this section, we present and analyze the results obtained by simulating the behavior of the vehicle driving through a single curve. This specific scenario was deliberately chosen to provide a clear and focused evaluation of the robustness and accuracy of our optimization algorithm within a controlled and relatively simple environment. The simplicity of this scenario allows us to isolate the key variables and better understand how the algorithm performs under specific conditions.

Before analysing the results, it is essential to define the parameters set for the simulation, as they play a crucial role in determining the outcome. Table 5.1 details the values of several adjustable parameters that significantly influence the behavior of the simulation. Of particular importance are the discretization step dsk_col , the degree of the interpolation polynomial d_th , which affects the smoothness and accuracy of the trajectory approximation, and the initial value for the longitudinal velocity v_{x0} that we imposed, which sets the starting speed of the vehicle as it enters the curve.

dsk_col [m]	d_th	$v_{x,0}$ [$\frac{m}{s}$]
30	3	1

Table 5.1. Parameters for the optimization process

In this specific scenario, we selected a value for the discretization step of dsk_col of 30 m , which represents the distance between consecutive grid points along the curve. This parameter is crucial as it directly influences the precision of the solution; a smaller discretization step typically results in higher accuracy by capturing more detail along the path. By conducting the initial simulation on a relatively short curve (significantly shorter than a full circuit), we can afford to use a smaller value for dsk_col . This approach enhances the accuracy of our results by providing a finer resolution of the vehicle's trajectory. Additionally, we set the initial longitudinal velocity to $v_{x,0} = 1 \frac{m}{s}$ to simulate the car starting from a standstill. However, we avoided setting the velocity precisely to zero, as this could lead to numerical instability or issues within the optimization process.

Another crucial factor for effectively solving the optimization problem is the definition of an initial guess vector of optimization variables which are the inputs to the solver. Notice that since we are addressing a multiple-shooting nonlinear constrained problem, the optimization variables must include both system's inputs and states. Therefore, it is essential to define an initial guess vector for both of them to guide the optimizer towards a correct and efficient solution.

In Table 5.2 are shown the initial guesses of the inputs and in Table 5.3 are shown the ones of the states.

$T_{drive} [Nm]$	$T_{brake} [Nm]$	$\Omega_z [\frac{rad}{s}]$
2000	-100	10^{-3}

Table 5.2. Parameters for the optimization process

$v_{x0} [\frac{m}{s}]$	$v_{y0} [\frac{m}{s}]$	$\Omega_{z0} [\frac{rad}{s}]$	$n_0 [m]$	$\epsilon_0 [rad]$	$\Omega_{f0} [\frac{rad}{s}]$	$\Omega_{r0} [\frac{rad}{s}]$
25	0	0	0	0	70.4225	70.4225

Table 5.3. Intial guess of the parameter for the optimization process on the single curve

Table 5.5 contains the values of the options that we used for the optimization procedure.

Figure 5.1 and Figures from 5.2 to 5.4 show the simulation results. We shall notice that the optimal trajectory is characterized by a compromise between smoothness (*i.e.*, curvature minimization) and distance minimization. In fact, Figure 5.1 highlights the attempt of the solver to widen the trajectory before engaging the first curve, while keeping the vehicle COG as close as possible to the track limits. This behaviour indeed resembles the one of real-world sports car. Figures 5.2, 5.3 and 5.4 confirm the premise that if the car starts from a standstill, there is no need for braking while navigating curves because the acceleration limits are within the friction ellipse. Consequently, T_{brk} is almost always close to zero, while T_{drv} remains consistently high. This behaviour is not observed for instance in the full-track lap time minimization.

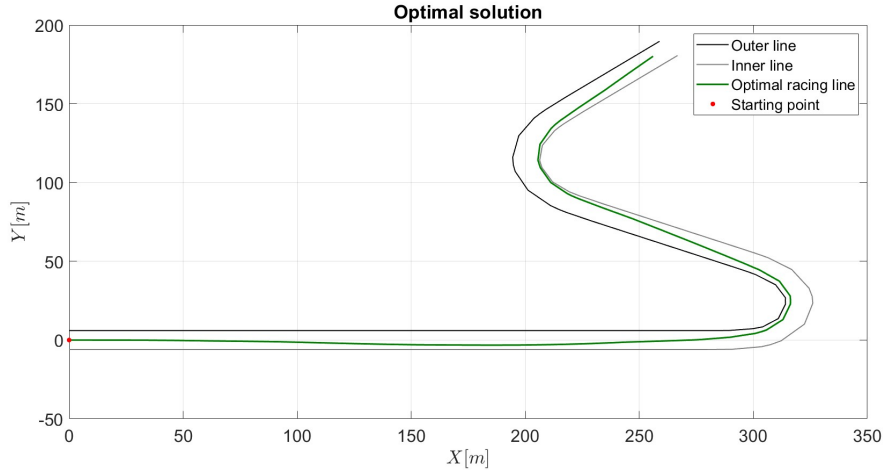


Figure 5.1: Optimal trajectory on a single curve obtained with $dsk_col = 30\text{ m}$

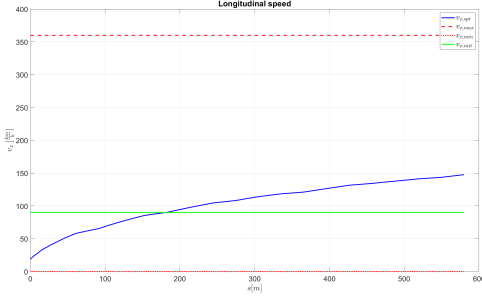


Figure 5.2: v_x evolution as function of the curvilinear abscissa

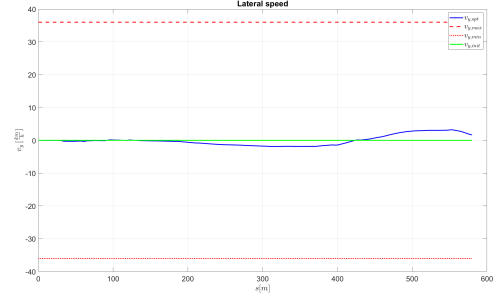


Figure 5.3: v_y evolution as function of the curvilinear abscissa

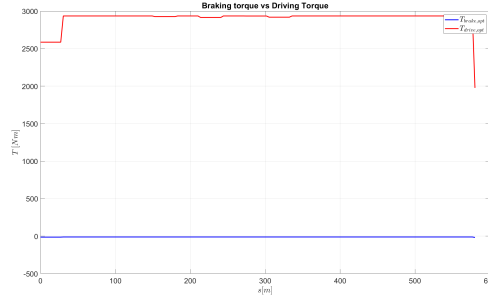


Figure 5.4: T_{brk} vs T_{drv} evolution as function of the curvilinear abscissa

5.0.2 Entire circuit

In this section, we present the results obtained from the simulation conducted on the entire circuit. To gain more insights, we ran multiple simulations with variations in some parameters. Among the parameters considered, the discretization step, denoted as dsk_col , plays a crucial role in maintaining the precision of our simulations.

Table 5.4 displays the values for several key parameters used in the simulation. These include the discretization step dsk_col , the degree of the interpolating polynomial d_th and the initial value for the longitudinal velocity v_{x0} . Additionally, Table 5.5 details the options configured for the optimizer during the first simulation while in Table 5.6 and Table 5.7 are shown respectively the initial guess for inputs and states variables for the optimization problem on the entire circuit. We shall highlight a high value of the c coefficient for the back tracking algorithm and a high value for the constraint tolerance. Both choices are associated with the high dimensionality of the problem, that involves a huge number of optimization variables. The former parameter setting was motivated by the fact that, if we accept a slight violation of the constraint (which is compatible with the degree of precision that our model captures), it is possible to achieve an acceptable solution in a reasonable time.

dsk_col [m]	d_th	v_{x0} [$\frac{m}{s}$]
70	3	1

Table 5.4. Parameters for the optimization process

Parameter	Value	Description
Hessmethod	BFGS	BFGS method
gradmethod	Central difference	Numerical method for gradient estimation
tolgrad	10^{-6}	Termination tolerance on the norm of the directional derivative
tolconstr	1	Constraint satisfaction tolerance
tolx	10^{-12}	Termination tolerance on the rel. change of opt. variables
tolfun	10^{-6}	Termination tolerance on the rel. improvement of the cost function
nitermax	1000	Termination tolerance on the number of iterations
ls_tkmax	1	Maximum step size
ls_beta	0.8	Beta scaling factor for back-tracking line search
ls_c	0.99	c coefficient factor for back-tracking line search
ls_nitermax	500	Maximum number of back-tracking line search iterations
Graddx	$2 \cdot 10^{-17}$	Step size for gradient calculation
BFGS_gamma	10^{-1}	γ factor for Powell's trick in BFGS algorithm

Table 5.5. List of options used by the optimizer

$v_{x0} [\frac{m}{s}]$	$v_{y0} [\frac{m}{s}]$	$\Omega_{z0} [\frac{rad}{s}]$	$n_0 [m]$	$\epsilon_0 [rad]$	$\Omega_{f0} [\frac{rad}{s}]$	$\Omega_{r0} [\frac{rad}{s}]$
25	0	0	0	0	70.4225	70.4225

Table 5.6. Intial guess of the parameter for the optimization process on the entire model

$T_{drive} [Nm]$	$T_{brake} [Nm]$	$\Omega_z [\frac{rad}{s}]$
2000	-100	10^{-3}

Table 5.7. Parameters for the optimization process

In Figure 5.5 the optimal trajectory of the racing car on the Yas Marina circuit is shown, which is the result of the optimization process. Also in this case, it is possible to notice the compromise between curvature and traveling distance minimization, with the vehicle that widens the trajectory before tight curves. Figures from 5.6 to 5.8 contain the evolution of car inputs T_{drv} , T_{brk} and δ along the track. It is noteworthy the jagged behaviour of the control variables, especially the steering angle, due to the absence of regularization terms in our problem formulation.

In Figures from 5.9 to 5.15 the state variables of the car are plotted, which are simulated and derived from vehicle's dynamic model. We can notice the relation between the torques applied to the wheels (Figure 5.6 and Figure 5.7) and the corresponding behaviour of the longitudinal velocity (Figure 5.9), front wheel angular velocity (Figure 5.14) and rear wheel angular velocity (Figure 5.15).

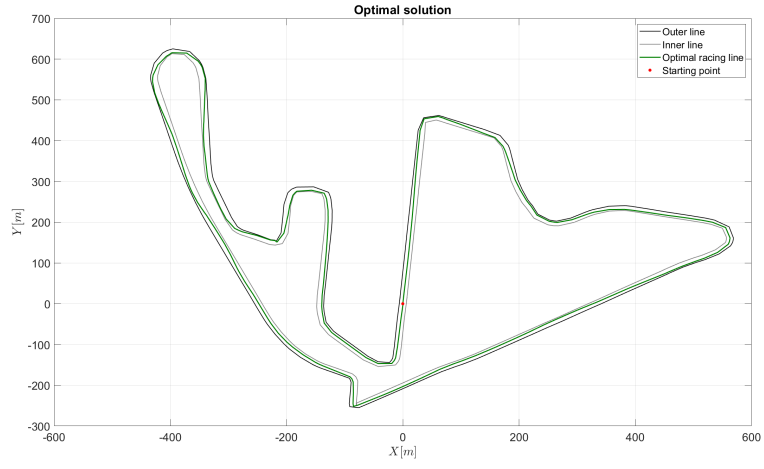


Figure 5.5: Optimal trajectory obtained for $dsk_col = 70\text{ m}$

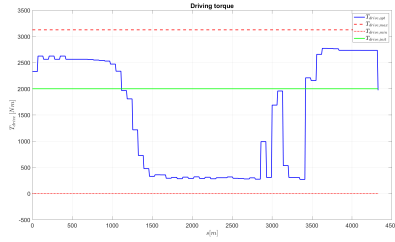


Figure 5.6: T_{drv} evolution as function of the curvilinear abscissa

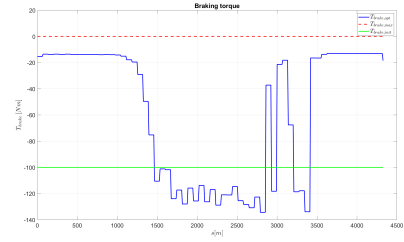


Figure 5.7: T_{brk} evolution as function of the curvilinear abscissa

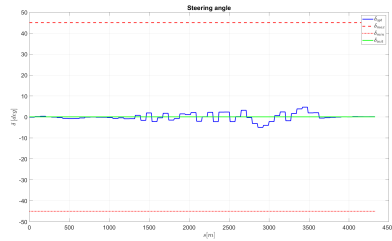


Figure 5.8: δ evolution as function of the curvilinear abscissa

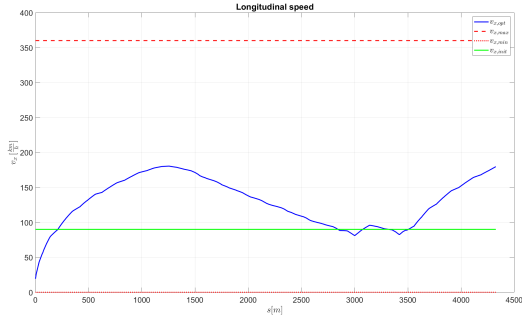


Figure 5.9: v_x evolution as function of the curvilinear abscissa

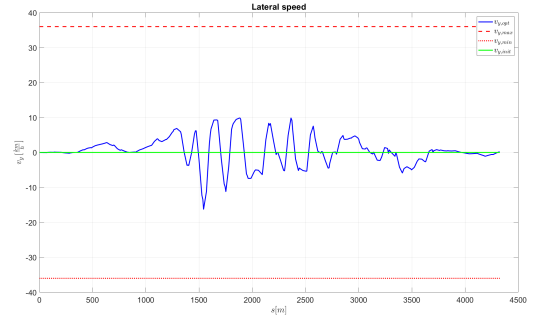


Figure 5.10: v_y evolution as function of the curvilinear abscissa

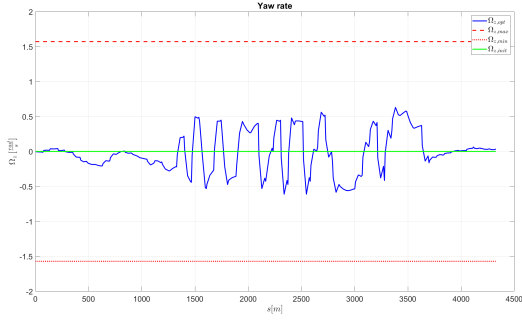


Figure 5.11: Ω_z evolution as function of the curvilinear abscissa

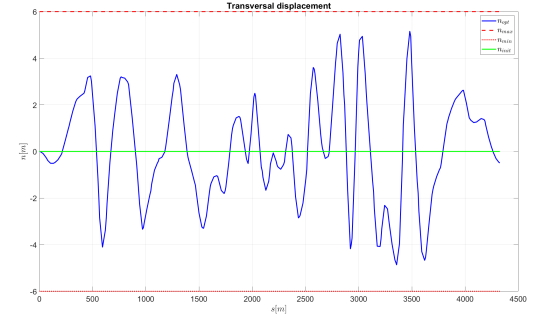


Figure 5.12: n evolution as function of the curvilinear abscissa

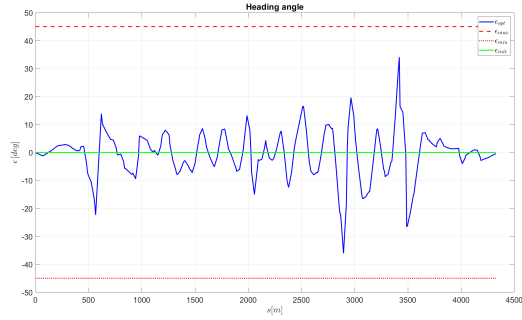


Figure 5.13: ϵ evolution as function of the curvilinear abscissa

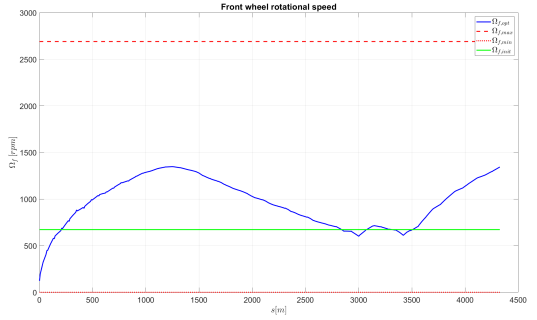


Figure 5.14: Ω_f evolution as function of the curvilinear abscissa

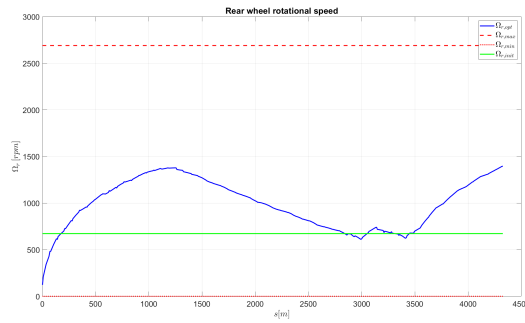


Figure 5.15: Ω_r evolution as function of the curvilinear abscissa

We will now proceed by performing another simulation with a reduced value of the dsk_col parameter. This modification will be implemented while keeping all other parameters and options unchanged (see Table 5.5, Table 5.7 and Table 5.6). In Table 5.8 the new values for the parameters of the optimization process are shown.

dsk_col [m]	d_th	v_{x0} [$\frac{m}{s}$]
50	3	10^{-2}

Table 5.8. Parameters for the optimization process

The result of the second simulation is shown in Figure 5.16

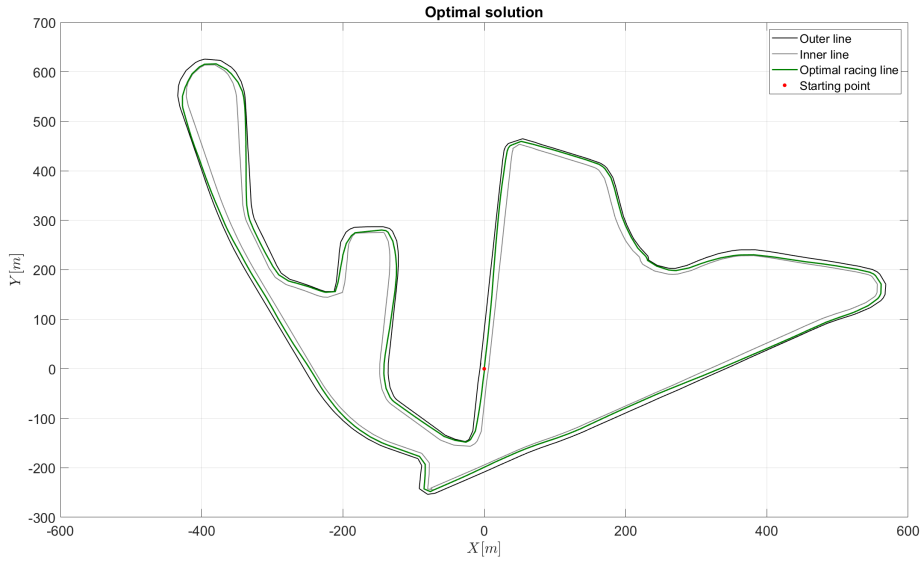


Figure 5.16: Optimal trajectory obtained for $dsk_col = 50\text{ m}$

Comparing Figure 5.5 and Figure 5.16 it is possible to notice the smoothing of the trajectory with the decreasing of the discretization step dsk_col . The price has to be paid in term of computational effort: while the solution with a discretization step of 70 m required ~ 9 minutes on a generic laptop, the discretization step of 50 m took almost ~ 60 minutes to converge to the final solution.

The solution to the optimization problem also shows a lack of smoothness. To obtain a smoother solution, we need to apply a technique known as *regularization* to the system inputs, which are T_{brk} , T_{drv} , and δ . Regularization involves incorporating a term into the objective function that is proportional to the derivative of these inputs. This term helps to penalize non-smooth solutions and encourages smoother trajectories. The overall objective function is expressed as follows:

$$J = \sum_{k=1}^N [\mathbf{B} \cdot \mathbf{Sf}_k \cdot ds_k + r_u \cdot U_k^2 + r_{du} \cdot \Delta U_k^2 + r_{du2} \Delta U_k^2] \quad (5.1)$$

where $U_k = [\delta_k, T_{drive}, T_{brake}]$ represents the vector of system inputs.

While this regularization method is quite advanced and effective in producing smoother solutions, we chose not to implement it. The difficulty lies in determining the appropriate values for the weighted vectors r_u , r_{du} , and r_{du2} , which makes the tuning process very complex.

Chapter 6

Discussion

After presenting the results of our work in detail, we will briefly conclude by analyzing the performance of our optimizer and describing possible uses and future developments.

6.1 Initial condition

The iterative method used by the NLP solver requires an initial guess for the decision variables. In some cases, a simple linear interpolation between the initial and final states can serve as a good guess, but for more complex systems, a more informed guess is usually necessary. When initial guesses are difficult to determine, a common approach is to solve a simplified version of the problem and use the solution as a starting point for the complete problem. When using the solution of the 'fast' problem as the initial guess of the 'final' problem, the solver could not converge. One hypothesis to explain this is that, even though the solution of the fast and final problems might seem similar when comparing the variables one by one, the combination of values found in the fast problem might not be feasible for the complete problem. Also, even though the initial guess may be closer to the optimal solution, it might be located in points of the space of the objective function from which the solver cannot converge to the optimal solution. In the end, the selected approach has been to initialise all the decision variables with 'sensible' constant values. For example, giving a value for the longitudinal velocity that is close to the average speed of the final solution that we expected. Using a value for the rotational speed of the wheels that is coherent with the longitudinal speed of the vehicle also has a noticeable effect on the performance of the solver.

6.2 Solver Performance

The performance of our solver is mainly affected by two factors: the distance at which we impose the collocation step and the tolerance we impose on the constraints, as we have a large number of equality constraints that make solver's task very difficult. Changing these values can cause the resolution time to range from a few minutes to hours, if not days.

However, since our model is not sufficiently accurate to perfectly describe the dynamics of a four-wheel vehicle, we decided to choose a compromise between solution quality and speed. To obtain better results in terms of solver performance while maintaining or even increasing the same quality, we should resort to external solvers to Matlab to solve the Quadratic Programming sub-problems.

Additionally, providing the Hessian matrix calculated manually would significantly help our solver; however, this option is impractical in our case due to the high number of optimization variables in our model.

6.3 Application-related results

First of all, our work can be improved by adding equations that account for load transfer and rolling resistance. We chose not to overcomplicate this solution because it is well-known from theory, as well as from practical experience, that the single-track model is not convenient for accounting these effects. For example, using a double-track model would permit to include load transfer at the cost of largely increase the optimization variables.

What is commonly done in practice is to use a single-track model or even a point mass to optimize the trajectory, and then apply a complex simulation model (such as a multi-body) to follow the generated trajectory and compute state variables.

Another improvement would be to reduce the constraint satisfaction tolerance parameter *tolconstr*. For the sake of computation time we decided to keep it quite high knowingly. Therefore, our results should be interpreted in light of these considerations.

Chapter 7

Conclusions

In this study, we analysed and solved the Minimum Lap Time Problem for an autonomous car on the Yas Marina circuit. To accomplish optimal performance in lap time simulation, a dynamic approach based on a rear wheel drive, single-track vehicle model was preferred over a geometric approach. The model permitted the incorporation of friction ellipse constraints based on sophisticated tire-road forces. Aspects such as the dynamic load transfer and rolling resistance have been neglected in the development of the solution, and can represent a further development.

Our analysis was based on the use of *direct orthogonal collocation* for the transformation of the problem from an Optimal Control Problem to a NonLinear Program. Other aspects such as scaling of the optimization variables and the selection of important parameters (such as the discretization step and the constraints tolerance) were carefully addressed. The SQP algorithm assisted with BFGS for hessian approximation successfully led to an optimal trajectory and vehicle state evolution.

Appendix

7.1 Appendix A: track equations

Equations for the track are derived here, following the developments in [2]. With reference to figure 7.1, we will define the vehicle's coordinate along the track in terms of its curvilinear abscissa s , defined as the distance traveled from the vehicle Center of Gravity from the starting line to the finish line, and the transversal displacement n . Also, ϵ will define the heading angle of the vehicle. The direction of n is indicated by vector $\vec{n}$, as in Figure 7.1, while $\vec{t}$ represents the tangent direction to the centerline.

Defining κ as the track curvature and ϑ_{CL} as the track centerline orientation angle, it is possible to write:

$$d\vartheta_{CL} = \kappa ds \quad (7.1)$$

Deriving (7.1) with respect to time:

$$\dot{\vartheta}_{CL} = \kappa \dot{s} \quad (7.2)$$

The vehicle's COG velocity along the $\vec{t}$ direction can be expressed as:

$$v_{COG}^{(\vec{t})} = v_x \cos(\epsilon) - v_y \sin(\epsilon) \quad (7.3)$$

Additionally if we define as $dx_{COG}^{(\vec{t})}$ the infinitesimal displacement along the $\vec{t}$ direction for the vehicle, we can write:

$$dx_{COG}^{(\vec{t})} = ds - n d(\vartheta_{CL}) \xrightarrow{\frac{d}{dt}} v_{COG}^{(\vec{t})} = \dot{s} - n \dot{\vartheta}_{CL} \quad (7.4)$$

Substituting (7.2) in (7.4) and equating the resulting expression with (7.3) leads to:

$$\dot{s} - n \kappa \dot{s} = v_x \cos(\epsilon) - v_y \sin(\epsilon) \quad (7.5)$$

Isolating $\dot{s}$ in (7.5) directly leads to (2.1).

Similarly, the COG velocity along the $\vec{n}$ can be both expressed as function of the transversal displacement n :

$$v_{COG}^{(\vec{n})} = \dot{n} \quad (7.6)$$

and in terms of v_x and v_y :

$$v_{COG}^{(\vec{n})} = v_x \sin(\epsilon) + v_y \cos(\epsilon) \quad (7.7)$$

Equating (7.6) and (7.7) leads to (2.2).

Conclusively, (2.3) can be obtained straightforwardly by deriving with respect to time the relationship (see figure 7.1):

$$\epsilon = \psi_z - \vartheta_{CL} \quad (7.8)$$

where ψ_z is the yaw angle, and substituting $\dot{\vartheta}_{CL}$ with expression (7.2).



References

- [1] Hans B. Pacejka. “Chapter 1 - Tire Characteristics and Vehicle Handling and Stability”. In: *Tire and Vehicle Dynamics (Third Edition)*. Ed. by Hans B. Pacejka. Third Edition. Oxford: Butterworth-Heinemann, 2012, pp. 1–58. ISBN: 978-0-08-097016-5. DOI: <https://doi.org/10.1016/B978-0-08-097016-5.00001-2>. URL: <https://www.sciencedirect.com/science/article/pii/B9780080970165000012>.
- [2] Giacomo Perantoni and David J.N. Limebeer. “Optimal control for a Formula One car with variable parameters”. In: *Vehicle System Dynamics* 52.5 (2014), pp. 653–678. DOI: [10.1080/00423114.2014.889315](https://doi.org/10.1080/00423114.2014.889315). eprint: <https://doi.org/10.1080/00423114.2014.889315>. URL: <https://doi.org/10.1080/00423114.2014.889315>.
- [3] Francesco Biral Nicola Dal Bianco Enrico Bertolazzi and Matteo Massaro. “Comparison of direct and indirect methods for minimum lap time optimal control problems”. In: *Vehicle System Dynamics* 57.5 (2019), pp. 665–696. DOI: [10.1080/00423114.2018.1480048](https://doi.org/10.1080/00423114.2018.1480048). eprint: <https://doi.org/10.1080/00423114.2018.1480048>. URL: <https://doi.org/10.1080/00423114.2018.1480048>.

Nomenclature

List of symbols

κ - centerline curvature

w - tracks width

s - curvilinear absissa

n - normal distance between the car and the centerline

ϵ - angle between the car's direction of movement and the tangent to the centerline

v_x - longitudinal velocity of the vehicle in its own reference frame

v_y - lateral velocity of the vehicle in its own reference frame

Ω_z - vehicle's yaw rate

t - time variable

z - generic state

u - generic input

$F_{x,f}$ - longitudinal force at the front tire in its local frame

$F_{y,f}$ - lateral force at the front tire in its local frame

$F_{x,r}$ - longitudinal force at the rear tire in its local frame

$F_{y,r}$ - lateral force at the rear tire in its local frame

F_{drag} - aerodynamic drag force

δ - front wheels steering angle

$F_{z,f}$ - front vertical load

$F_{z,r}$ - rear vertical load

F_{z0} - nominal vehicle load

l_f - longitudinal distance from the center of gravity of the vehicle to the front axle

l_r - longitudinal distance from the center of gravity of the vehicle to the rear axle

l - vehicle wheelbase

M - vehicle's mass

I_z - vehicle's moment of inertia around its local z-axis

Ω_f - front wheel angular velocity

Ω_r - rear wheel angular velocity

T_r - applied torque to the rear wheels

T_f - applied torque to the front wheels

R_w - wheel radius

J_w - wheel rotational inertia

F_{lift} - aerodynamic downforce

g - gravity acceleration

$D_{x,f}$ - friction-dependent force in Pacejka's Magic Formula for front longitudinal force

$D_{x,r}$ - friction-dependent force in Pacejka's Magic Formula for rear longitudinal force

c_x, b_x, e_x - experimental coefficients in Pacejka's Magic Formula for longitudinal forces

$D_{y,f}$ - friction-dependent force in Pacejka's Magic Formula for front lateral force

$D_{y,r}$ - friction-dependent force in Pacejka's Magic Formula for rear lateral force

c_y, b_y, e_y - experimental coefficients in Pacejka's Magic Formula for lateral forces

μ_x - longitudinal friction coefficient

μ_y - lateral friction coefficient

μ_0 - friction coefficient at nominal load

p_{D2} - negative tire load sensitivity coefficient

λ_f - front wheel slip angle

λ_r - rear wheel slip angle

$s_{x,f}$ - front longitudinal slip ratio

$s_{x,r}$ - rear longitudinal slip ratio

$v_{x,r}$ - longitudinal speed of the rear wheel

$v_{x,f}$ - longitudinal speed of the front wheel

$F_{roll,f}$ - front rolling resistance

$F_{roll,r}$ - rear rolling resistance

C_d - drag coefficient
 C_l - lift coefficient
 ρ_{air} - air density
 A_f - vehicle front area
 T_{brk} - total brake torque applied to the vehicle
 K_{brk} - braking torque allocation coefficient
 T_{drv} - total drive torque applied to the vehicle
 T_f - torque applied on the front axle of the vehicle
 T_r - torque applied on the rear axle of the vehicle
 J - Minimum Lap Time Problem cost function
 x - generic optimization variable
 t_f - final time of the Minimum Lap Time Problem
 s_f - final distance of the Minimum Lap Time Problem
 t_k - time instant in the entire time domain
 d_{th} - degree of interpolating polynomial
 p_k - k-th interpolating Lagrange polynomial
 v_k - k-th polynomial interpolating coefficients
 $l_{k,i}$ - basis polynomials at i-th point in the k-th interval in *direct orthogonal local collocation*
 $f(\cdot)$ - cost function of a generic optimization problem
 $g(\cdot)$ - equality constraint function of a generic optimization problem
 $h(\cdot)$ - inequality constraint function of a generic optimization problem
 x^* - local minimizer
 Ω - feasible set
 T_Ω - tangent cone to a feasible set
 A - active set
 λ^*, μ^* - Lagrange multipliers
 γ - BFGS tuning parameter
 y - BFGS Lagrangian difference over two successive steps
 $\bar{s}$ - BFGS optimization variable difference over two successive steps

$T_1(\cdot)$ - ℓ_1 -norm merit function

σ_i, τ_i - parameters in the ℓ_1 -norm merit function

$L(\cdot)$ - Lagrangian function

η - step size in the Central Finite Difference

$\vec{\mathbf{t}}$ - unity vector tangent to the centerline

$\vec{\mathbf{n}}$ - unity vector normal to the centerline

ϑ_{CL} - track centerline orientation angle

$\mathbf{p}$ - searching direction

$\bar{t}_k$ - step size in the BFGS algorithm

List of Acronyms

MLTP - Minimum Lap Time Problem

SQP - Sequential Quadratic Programming

COG - Center Of Gravity

OCP - Optimal Control Problem

ODE - Ordinary Differential Equation

NLP - NonLinear Program

LICQ - Linearly Independent Constraint Qualification

QP - Quadratic Programming